

1. Texto

Farei aqui uma breve análise do algoritmo de busca linear contra busca binária

Sempre será buscado o último item da lista, que será o pior caso para ambos os algoritmos, para testar com efetividade

Além disso, para cálculo de tempo, foi classe `timeit` da biblioteca `timeit` que, recebe uma função como parametro e um numero inteiro, ela roda aquela função o numero de vezes que voce passou nesse inteiro e retorna o tempo total de execução. Assim podemos calcular a média dessas execuções e chegar a um valor mais preciso, do que apenas testar a função uma unica vez.

Digo ainda, que, para fins de teste, admitisse que os vetores já estão ordenados e contem apenas um tipo de dados (Números inteiros)

A analise englobará 2 graficos e uma tabela

GRÁFICO 1

TEMPO X TAMANHO DA LISTA (MOSTRARÁ A CURVA DE CRESCIMENTO DE TEMPO DE EXECUÇÃO COM AMBOS ALGORTIMOS)

GRÁFICO 2

INTERAÇÕES X TAMANHO DA LISTA (MOSTRA A CURVA EM RELACIONADA A NOTAÇÃO BIG O)

** Para os gráficos 1 e 2 o eixo x terá 7 tamanhos diferentes de lista, começando em 10 elementos e terminando em 10 milhões de elementos

Tabela de comparação por posição

(MOSTRA EM QUE MOMENTO UM ALGORITMO PASSA A SER MELHOR QUE O OUTRO

□ Para esta tabela o tamanho da lista será fixo em 10000 elementos

O intuito aqui é visualizar em que momento um algoritmo passa a ter vantagem sobre outro além de mensurar essa vantagem.

Formato da tabela:

posição	linear	binária	vantagem
1	0.3 ms	0.5 ms	linear 18×
10	0.5 ms	1.5 ms	linear 11×
100	2 ms	1.75 ms	linear 2.7×
1000	20 ms	2.5 ms	binária 3.6×

2. Código

```
#=====CÓDIGO DE BUSCA LINEAR E BINÁRIA=====
import timeit

#BENCHMARK SEARCHING CONFIGURATIONS-----
def get_last_value(vetor:list[int]) -> int:
    """
    Função para pegar o último valor da lista,
    Assim testando os algoritmos no pior dos casos!
    """
    return vetor[-1]

listt_len = [10,100,1000,10000,100000,
             1_000_000,10_000_000] #Teste com varios tamanhos de lista

listt_position =[1,10,50,100,500,1000] #Teste de busca em varias posições da lista


def get_interations(vetor:list[int],value:int) -> tuple[int,int]:
    """
    Função para pegar o numero de interações quando busca x valor em y vetor
    para cada um dos algoritmos
    """
    iter_binary = binary_search(vetor,value)[1]
    iter_linear = linear_search(vetor,value)[1]
    return (iter_linear,iter_binary)

#-----

#funções de bbusca
def binary_search(vetor:list[int],value:int) -> tuple[int|str,int]:
    """
    manual implementation of a binary search for studies
    """

    down = 0
    upper = len(vetor)-1
    middle = (upper + down) // 2

    interations_count = 0

    while down <= upper:
        interations_count += 1
        if value == vetor[middle]:
            return (middle,interations_count)

        elif value > vetor[middle]:
            down = middle + 1
        elif value < vetor[middle]:
            upper = middle -1
        middle = (upper + down) // 2

    return ("Valor não existe no vetor",interations_count)

def linear_search(vetor:list[int],value:int) -> tuple[int|str,int]:
    interations_count = 0
    for n,item in enumerate(vetor,0):
        interations_count += 1
        if value == item:
```

```

        return (n,iterations_count)
    return ("Valor não existe no vetor",iterations_count)

#=====

if __name__ == "__main__":
    import matplotlib.pyplot as plt

    dict_len_x_Time_and_iter = {}
    for tamanho in listt_len:

        dict_len_x_Time_and_iter[tamanho] = {}
        listt = [i for i in range(1,tamanho+1)]
        search_value = get_last_value(listt)

        tempo_binary = timeit.timeit(
            lambda: binary_search(listt, search_value),
            number=10
        )/10
        tempo_linear = timeit.timeit(
            lambda: linear_search(listt, search_value),
            number=10
        )/10
        iter_tuple = get_interactions(listt,search_value)

        dict_len_x_Time_and_iter[tamanho]['Tempo linear'] = tempo_linear * 1000
        dict_len_x_Time_and_iter[tamanho]['Tempo binário'] = tempo_binary * 1000
        dict_len_x_Time_and_iter[tamanho]['Interações binário'] = iter_tuple[1]
        dict_len_x_Time_and_iter[tamanho]['interações linear'] = iter_tuple[0]

    for tamanho, estatis in dict_len_x_Time_and_iter.items():
        print(
            f"TAMANHO: {tamanho}",
            f" Tempo linear: {estatis['Tempo linear']:.5f} ms",
            f" Tempo binário: {estatis['Tempo binário']:.5f} ms",
            f" Interações linear: {estatis['interações linear']}",
            f" Interações binário: {estatis['Interações binário']}\n",
            sep="\n"
        )

    lista_tempo_linear = [y['Tempo linear'] for y in dict_len_x_Time_and_iter.values()]
    lista_tempo_bin = [y['Tempo binário'] for y in dict_len_x_Time_and_iter.values()]
    lista_iter_linear = [y['interações linear'] for y in dict_len_x_Time_and_iter.values()]
    lista_iter_bin = [y['Interações binário'] for y in dict_len_x_Time_and_iter.values()]

    listt_len = list(dict_len_x_Time_and_iter.keys())

#Gráfico 1 =====
plt.figure(figsize=(10, 6))
plt.plot(listt_len,lista_tempo_bin,label="Busca binária")
plt.plot(listt_len,lista_tempo_linear,label="Busca linear")

plt.title("Tempo de execução x Tamanho da lista")
plt.xlabel("Tamanho da lista")
plt.ylabel("Tempo médio (ms)")

plt.legend()
plt.grid(alpha=0.3)
plt.xscale("log")
plt.xticks(
    listt_len,
    ["10", "100", "1K", "10K", "100K", "1M", "10M"]
)

#Gráfico 2 =====

plt.figure(figsize=(10, 6))

```

```

plt.plot(
    listt_len,
    lista_iter_linear,
    marker="o",
    label="Busca linear"
)

plt.plot(
    listt_len,
    lista_iter_bin,
    marker="o",
    label="Busca binária"
)

plt.title("Iterações x Tamanho da lista")
plt.xlabel("Tamanho da lista")
plt.ylabel("Número de iterações")

plt.xscale("log")
plt.yscale("log")

plt.yticks(
    listt_len,
    ["10", "100", "1K", "10K", "100K", "1M", "10M"]
)

plt.xticks(
    listt_len,
    ["10", "100", "1K", "10K", "100K", "1M", "10M"]
)

plt.legend()
plt.grid(alpha=0.3)

plt.show()

```

3. Plots e tabela

Gráfico 1

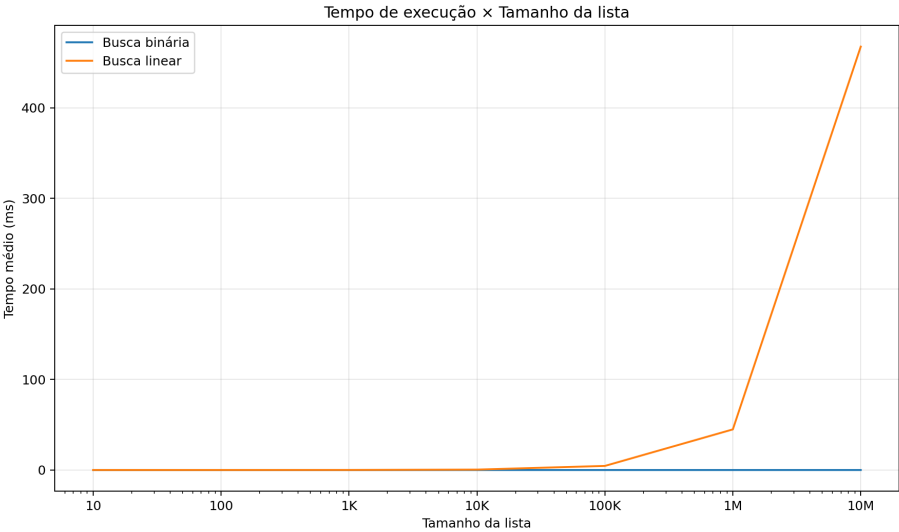


Gráfico 2

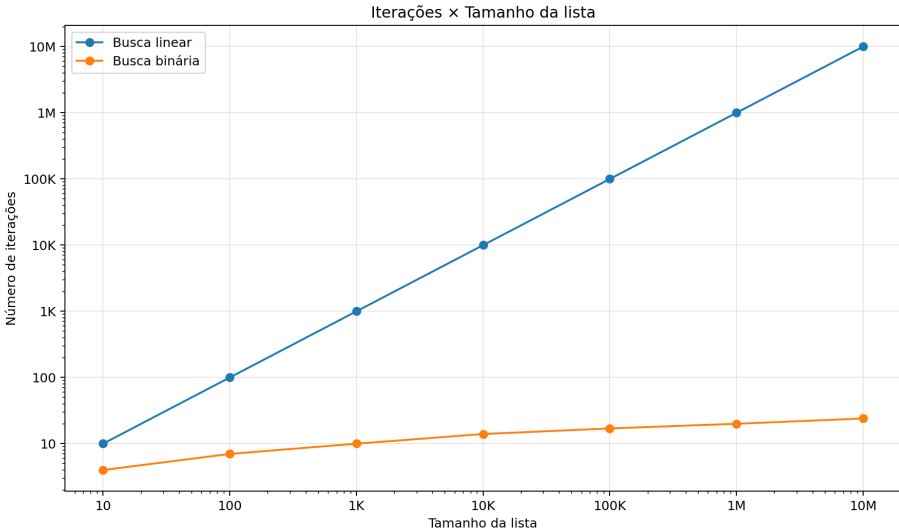


Tabela de comparação por posição

posição	linear (ms)	binária (ms)	vantagem
1	0.000158	0.001470	linear 9.29×
10	0.000408	0.001412	linear 3.46×
50	0.001511	0.001375	binária 1.10×
100	0.002944	0.001356	binária 2.17×
500	0.018665	0.001535	binária 12.16×
1000	0.043506	0.001287	binária 33.79×